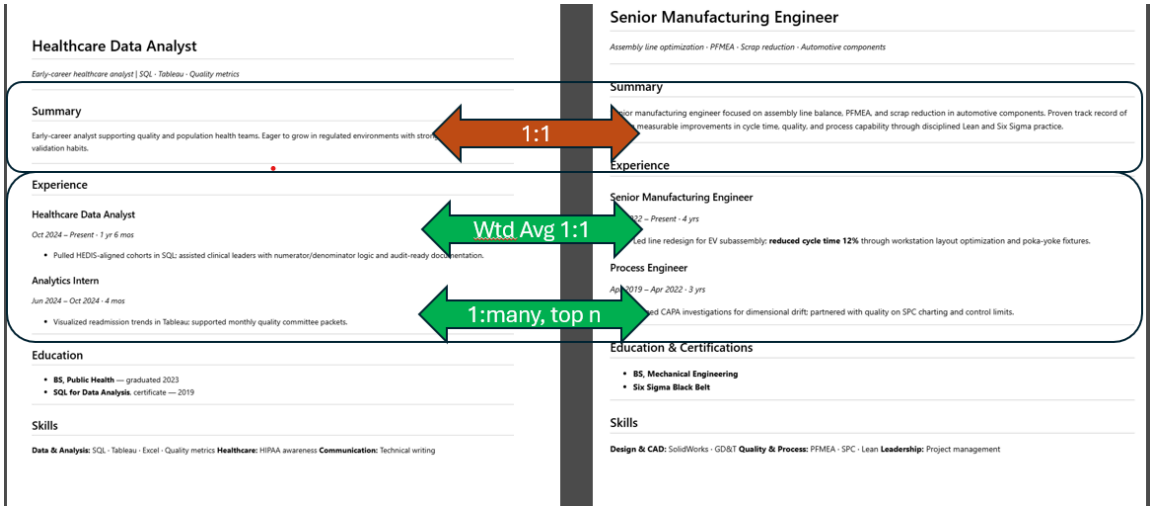
Resume comparison: "Who is most like me"



Get resumes in consumable format

- (see build_resume_corpus.ipynb)

ResumeID	Section	Title	ItemID	Text	MonthsDuration	RecencyMonthsAgo
2	Summary	BI Analyst	1	Retail analytics professional who turns store, e-commerce, and loyalty data into clear merchandising and inventory decisions. Comfortable owning dashboards end-to-end from requirements through production refresh.		
2	Experience	Senior BI Analyst	2	Built executive sales dashboards in Power BI; standardized KPI definitions across regions and cut ad-hoc report load for finance partners.	36	0
2	Experience	Reporting Analyst	3	Automated weekly inventory exception reports in SQL; partnered with supply chain to reduce stockouts on top SKUs.	24	36
2	Education		4	BS Business Analytics		36
2	Education		5	AS Business Administration		84
2	Skills		6	SQL		
2	Skills		7	Power BI		
2	Skills		8	DAX		
2	Skills		9	Excel		
2	Skills		10	Retail operations		
2	Skills		11	Stakeholder communication		

Create Embeddings

ResumeID	Section	Title	ItemID	Text	MonthsDu	RecencyM	embedding
2	Summary	BI Analyst	1	Retail analytics professional who			[-0.04376695677638054, 0.0254958104342
2	Experience	Senior BI A	2	Built execu	36	0	[-0.013969175517559052, 0.069439649581
2	Experience	Reporting A	3	Automatec	24	36	[-0.03903426229953766, 0.013249041512

Step 1: Load resume csv file

- Loads `output/corpus_20_resumes.csv`, encodes **each row** in **Summary**, **Experience**, and **Skills** with `all-MiniLM-L6-v2`, adds an **`embedding`** column (JSON array per row), and writes **`output/corpus_with_embedding.csv`**.

- Set `TARGET_RESUME_ID` below to choose which resume is “me” for the optional ranking cell.

In [4]:
```
from __future__ import annotations

import json
from pathlib import Path

import numpy as np
import pandas as pd

_cwd = Path.cwd().resolve()
if (_cwd / "corpus_similarity.ipynb").exists():
    NOTEBOOK_DIR = _cwd
elif (_cwd / "peopleLikeMe" / "corpus_similarity.ipynb").exists():
    NOTEBOOK_DIR = _cwd / "peopleLikeMe"
else:
    NOTEBOOK_DIR = _cwd

OUTPUT_DIR = NOTEBOOK_DIR / "output"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

CORPUS_PATH = OUTPUT_DIR / "corpus_20_resumes.csv"
META_PATH = OUTPUT_DIR / "resume_metadata.csv"
OUT_EMB_PATH = OUTPUT_DIR / "corpus_with_embedding.csv"

TARGET_RESUME_ID = 1  # change to any ResumeID in the corpus

df = pd.read_csv(CORPUS_PATH, dtype={"ResumeID": int, "ItemID": int})
df["Title"] = df["Title"].fillna("").astype(str)

CORPUS_PATH.resolve(), len(df), sorted(df["ResumeID"].unique())[:5], "...", df["Sec
```

Out[4]:
```
(WindowsPath('C:/Users/DeloresMincarelli/Documents/Portfolio/peopleLikeMe/output/c
orpus_20_resumes.csv'),
 236,
 [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)],
 '...',
 ['Summary', 'Experience', 'Education', 'Skills'])
```

Step 2: Install Py sentence-transformers & Scikit-learn & Embedding model (it's free)

Sentence-transformers

- A library for turning sentences or short texts into fixed-size vectors (embeddings) so you can compare meaning, not just words.
- Built on PyTorch and Hugging Face Transformers. Pre-trained models (e.g. MiniLM, MPNet) map text to dense vectors; similar meanings tend to sit close in vector space. Common uses: semantic search, duplicate detection, clustering, re-ranking, similarity between documents or queries. It’s the usual choice when you want “what does this sentence mean?” style similarity without training a big model from scratch.

Scikit-learn

- The main Python library for classical / tabular machine learning and scientific computing around models.

Together: sentence-transformers gives you text embeddings; scikit-learn can use those vectors (or other features) for clustering, classification, nearest neighbors, etc.

Load embedding model: all-MiniLM-L6-v2 (from SentenceTransformers) because it's:

- Built specifically for semantic similarity It converts text → vectors (embeddings) Similar meaning → vectors close together (cosine similarity) That's exactly what your workflow needs: Resume bullets ↔ skills/tasks matching
- Very efficient Small model (~80MB) Fast on CPU (no GPU needed) Great for batching thousands of rows (like O*NET)
- "Good enough" accuracy
- Is it free? Yes — completely free Open-source (Apache 2.0 license) No API calls Runs locally No usage limits No cost per embedding That's why it's commonly used in: prototypes and offline pipelines

In [5]:
```
# pip install sentence-transformers scikit-learn

from sentence_transformers import SentenceTransformer

MODEL_NAME = "all-MiniLM-L6-v2"

model = SentenceTransformer(MODEL_NAME)

print("Model loaded successfully")
```

```
Loading weights:   0%|          | 0/103 [00:00<?, ?it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok
if you expect identical arch.
Model loaded successfully
```

Step 3: Encode Summary, Experience, and Skills (one embedding per row)

Education rows are kept in the export with an **empty** `embedding`. Vectors are stored as **JSON arrays** in the CSV.

In [6]:
```
ENCODE_SECTIONS = {"Summary", "Experience", "Skills"}
```

```python
def row_to_encode_text(row: pd.Series) -> str:
    section = str(row["Section"]).strip()
    text = str(row["Text"]).strip()
    title = str(row["Title"]).strip() if pd.notna(row["Title"]) else ""
    if section == "Summary":
        return text
    if section == "Experience":
        return f"{title}: {text}" if title else text
    if section == "Skills":
        return text
    return text


mask = df["Section"].isin(ENCODE_SECTIONS)
encode_idx = df.index[mask]
texts = [row_to_encode_text(df.loc[i]) for i in encode_idx]

vectors = model.encode(texts, show_progress_bar=True)

df_out = df.copy()
df_out["embedding"] = ""
for row_i, idx in enumerate(encode_idx):
    df_out.at[idx, "embedding"] = json.dumps(vectors[row_i].tolist())

df_out.to_csv(OUT_EMB_PATH, index=False, encoding="utf-8", na_rep="")

encoded_rows = int(mask.sum())
print("Wrote", OUT_EMB_PATH.resolve())
print("Encoded rows:", encoded_rows, "vector dim:", vectors.shape[1] if len(vectors
```

```
Batches:   0%|          | 0/7 [00:00<?, ?it/s]
Wrote C:\Users\DeloresMincarelli\Documents\Portfolio\peopleLikeMe\output\corpus_with
_embedding.csv
Encoded rows: 196 vector dim: 384
```

From ChatGPT: How to compare?

Core idea: A resume is a structured portfolio of meanings (summary + many bullets), not one document and not interchangeable snippets. “Professionals like me” depends on what you mean by similar (overall identity vs day-to-day work vs strongest specialties vs breadth), and no single number captures all of that.

What to avoid

- Treating “embed the embeddings” as a magic second step: if you collapse many vectors to one, that should be an explicit aggregation (mean, weighted mean, role/theme structure), not an opaque second embedding model.
- Do not rely on one max bullet match or one plain average over all bullets as the whole story: it's fragile, length-sensitive, and easy to critique.
- Letting summary similarity dominate: often generic and polished.

... Instead ...

- Multi-level comparison:
  - (1) summary (broad positioning),
  - (2) weighted bullet centroid (holistic work signal), and
  - (3) bullet-level alignment (concrete overlap), then a
  - (4) weighted composite (20% Summary / 30% Wtd Bullet / 50% Bullet Alignment).
- For bullets: bidirectional aggregation (A→B and B→A) so broad vs narrow profiles are not confused. Use top-k (or proportional k) over per-bullet best matches so a few strong lines and noise from long resumes do not dominate a naive full average.
- For scrutiny: add explainability (component scores + a few supported bullet pairs)
- The defensible story is: similarity is estimated at several levels, aggregated with clear rules, and reported with evidence—not "one cosine score proves sameness." That matches how embeddings are usually used in high-stakes settings: transparent signals, explicit limits on what the score means.

Step 4: Compare Summaries (broad positioning)

SummarySim between two resumes is the **cosine similarity** of their **Summary** row embeddings (already stored in `output/corpus_with_embedding.csv`). This cell reads that CSV only (no re-encode), uses `sklearn.metrics.pairwise.cosine_similarity`, ranks every resume against `TARGET_RESUME_ID`, and merges `resume_metadata.csv` for context - this has a high level description of the resume (ex: finance, senior level)

Summary text is often polished and uses generic phrases (*results-driven*, *cross-functional*, *data-driven*), so cosine similarity here can **overstate** how alike two people are in real work. Treat this as **one component** of a future composite score, not a full "professionals like me" verdict.

Checks: similarity of the target to itself should be **1.0** (within float noise). All similarities lie in **[-1, 1]**.

Cosine similarity measures the **angle** between two vectors, **ignoring their magnitude**

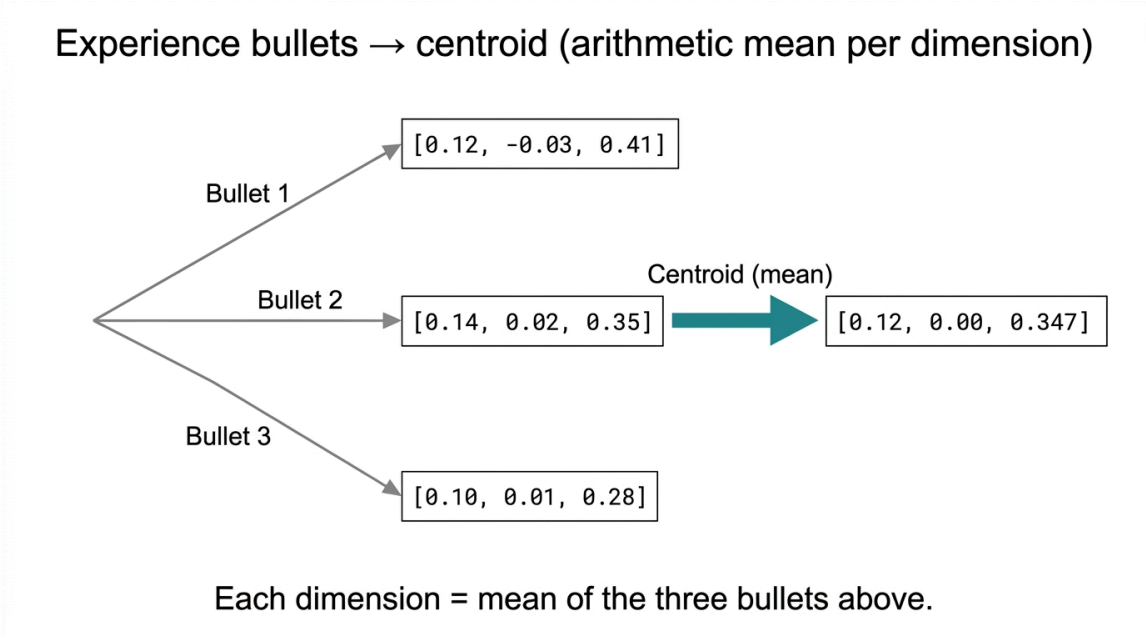
In [13]:
```
from sklearn.metrics.pairwise import cosine_similarity

if not OUT_EMB_PATH.is_file():
    raise FileNotFoundError(f"Embedded corpus not found: {OUT_EMB_PATH.resolve()}")
if not META_PATH.is_file():
    raise FileNotFoundError(f"Resume metadata not found: {META_PATH.resolve()}")

emb_df = pd.read_csv(OUT_EMB_PATH, dtype={"ResumeID": int, "ItemID": int})
sum_mask = emb_df["Section"].astype(str).str.strip() == "Summary"
sum_df = emb_df.loc[sum_mask & emb_df["embedding"].astype(str).str.len().gt(0)].cop

counts = sum_df.groupby("ResumeID").size()
all_ids = sorted(emb_df["ResumeID"].unique())
```

```
missing_summary = [rid for rid in all_ids if rid not in counts.index or counts.loc[
if missing_summary:
    raise ValueError(f"No non-empty Summary embedding for ResumeID(s): {missing_sum
multi = counts[counts > 1]
if len(multi):
    raise ValueError(
        "Expected exactly one Summary row per ResumeID; got duplicates: "
        f"{multi.to_dict()}"
    )

ids = sorted(sum_df["ResumeID"].unique())
if TARGET_RESUME_ID not in ids:
    raise ValueError(f"TARGET_RESUME_ID={TARGET_RESUME_ID} not in corpus summary ro

def summary_vec(rid: int) -> np.ndarray:
    row = sum_df.loc[sum_df["ResumeID"] == rid, "embedding"].iloc[0]
    return np.array(json.loads(row), dtype=np.float32)

vectors = np.stack([summary_vec(rid) for rid in ids], axis=0)
target_row = ids.index(TARGET_RESUME_ID)
target_vec = vectors[target_row].reshape(1, -1)

sims = cosine_similarity(target_vec, vectors)[0]
rank_df = (
    pd.DataFrame({"ResumeID": ids, "summary_cosine_similarity": sims.astype(float)}
    .sort_values("summary_cosine_similarity", ascending=False)
    .reset_index(drop=True)
)
meta = pd.read_csv(META_PATH)
rank_df = rank_df.merge(meta, on="ResumeID", how="left")

self_sim = float(sims[target_row])
assert abs(self_sim - 1.0) < 1e-5, f"expected target self-similarity ~1, got {self_
assert sims.min() >= -1.0001 and sims.max() <= 1.0001

rank_df
```

Out[13]:

	ResumeID	summary_cosine_similarity	primary_domain	role_family	seniority
0	1	1.000000	data_analytics	healthcare_analytics	
1	4	0.515133	data_analytics	healthcare_reporting	
2	7	0.499845	data_science	research_ds	
3	8	0.444134	data_science	nlp_search	
4	2	0.406697	data_analytics	bi_reporting	
5	9	0.361967	data_science	forecasting	
6	14	0.335894	finance	fpna	
7	3	0.330743	data_analytics	operations_analytics	
8	13	0.329266	hr	total_rewards	
9	17	0.265207	engineering	structural	
10	20	0.258688	marketing	brand_content	
11	5	0.257279	data_analytics	product_analytics	
12	11	0.254773	hr	talent_acquisition	
13	16	0.250858	finance	internal_audit	
14	10	0.240335	data_analytics	public_sector	
15	19	0.239304	marketing	performance_marketing	
16	6	0.233892	data_science	ml_engineering	
17	15	0.219156	finance	corporate_accounting	
18	12	0.150982	hr	employee_relations	
19	18	0.030978	engineering	manufacturing_quality	

Step 5: Weighted Avg of Experience bullets - holistic work signal



- What if Resume A has 10 bullets, B has 3 — what's the impact? Per resume: A's centroid uses only A's 10 vectors; B's uses only B's 3. Bullet counts do not get mixed when building each centroid. Still comparable: Both centroids live in the same embedding space, so cosine between C_A and C_B is well-defined.
- How count matters (indirectly):

More bullets → the centroid is an average over more directions. Diverse bullets can pull the average toward something broader / more "middle" (sometimes described as washing out sharp specialties). Many bullets saying similar things pull the centroid strongly toward that theme (especially if weights are similar). Fewer bullets → the centroid is dominated by those few directions; it can look more specific or noisier as a summary of the whole career, depending on what those three bullets say. So: 10 vs 3 does not change the mathematical type (always one 384-dim vector per resume). It changes how that vector is formed and therefore where it sits in space—and that in turn affects the cosine score when you compare A to B, but there is no separate "penalty" for 10 vs 3 in the centroid formula itself; the effect shows up through content and diversity of those bullets (and your recency weights).

CentroidSim compares each resume's **single centroid vector C_R**: a **weighted mean** of **Experience** row embeddings from `corpus_with_embedding.csv` (same dimension as one bullet, e.g. 384). **CentroidSim** vs `TARGET_RESUME_ID` is **cosine similarity** between **C_target** and each **C_R** via `sklearn.metrics.pairwise.cosine_similarity`.

Configurable (top of the next code cell): three weights `WEIGHT_EXPERIENCE_RECENT` / `MID` / `OLD`, and two month cutoffs `RECENCY_MONTHS_AGO_MAX_RECENT` and `RECENCY_MONTHS_AGO_MAX_MID` from `RecencyMonthsAgo` (**NaN** → recent tier). Invalid thresholds (`MAX_RECENT >= MAX_MID`) raise `ValueError`.

A centroid captures **broad semantic overlap** across bullets, but **averaging can wash out** sharp specialties; **many similar bullets** pull **C_R** toward that theme. Use this as **component 2** of a future composite, not a full similarity verdict.

Checks: target self-similarity **≈ 1**; all cosines in **[-1, 1]**. Spot-read Experience text for a few pairs against the ranking.

Example: What's actually happening

Original vector:
v = [1, 2, 3]
Scaled vector because this bullet is recent (configurable):
1.25 × v = [1.25, 2.5, 3.75]
Key geometric insight
The direction does NOT change
Only the length (magnitude) increases
The scaled vector lies on the same line from the origin
Now translate that to embeddings (this is the important part)

Think of an embedding like:

A direction in space = the meaning The length = how strongly that meaning is expressed (or weighted)

So… does scaling change meaning?

Short answer: 👉 No — scaling does NOT change the semantic meaning

Why:

Most embedding comparisons use cosine similarity
Cosine similarity depends only on angle, not magnitude

So:

v and 1.25·v
have identical cosine similarity (1.0)
→ interpreted as exact same meaning
Why scaling is still used (important for your centroid work)

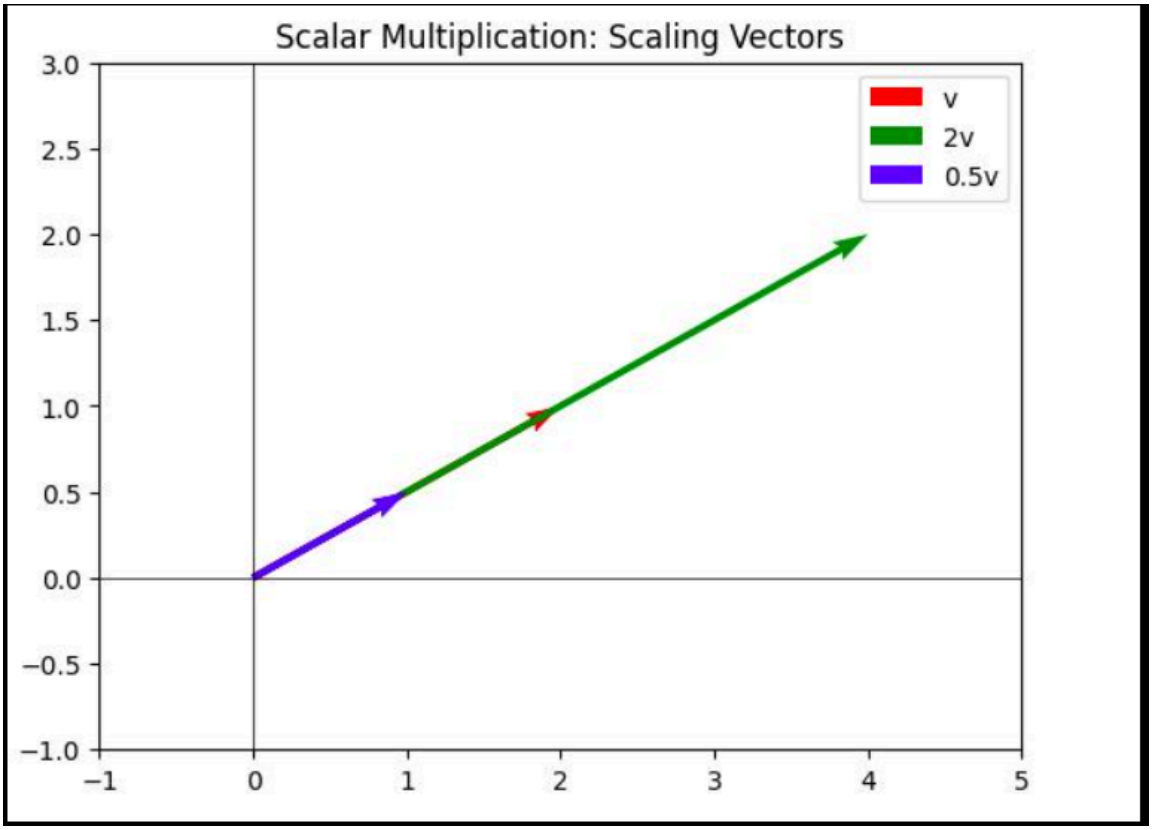
Scaling is not about changing meaning — it's about influence.

From your example:

Recent bullet → weight = 1.25
Older bullet → weight = 0.75

When you scale:

You're saying: 👉 "This vector should pull the centroid more"



Example

(your real vectors have length 384, but the steps are the same).

Setup — Resume A

Recent
$v_1 = [1, 2, 3]$
$\alpha_1 = 1.25$

Old
$v_2 = [1, 5, 7]$
$\alpha_2 = 0.75$

Step 1 — Scale each vector by its weight
$\alpha_1 v_1 = 1.25 \times [1, 2, 3] = [1.25, 2.5, 3.75]$

$\alpha_2 v_2 = 0.75 \times [1, 5, 7] = [0.75, 3.75, 5.25]$

Step 2 — Add the weighted vectors (component-wise)
$\alpha_1 v_1 + \alpha_2 v_2 = [1.25 + 0.75, 2.5 + 3.75, 3.75 + 5.25] = [2, 6.25, 9]$

Step 3 — Add the weights
$\alpha_1 + \alpha_2 = 1.25 + 0.75 = 2$

Step 4 — Divide to get the centroid (weighted average)
$C_A = (\alpha_1 v_1 + \alpha_2 v_2) / (\alpha_1 + \alpha_2) = [2, 6.25, 9] / 2 = [1, 3.125, 4.5]$

So Resume A's centroid is the 3-vector C_A = [1, 3.125, 4.5].
That is not a single number; it is one point in the same 3-D space as the bullets.

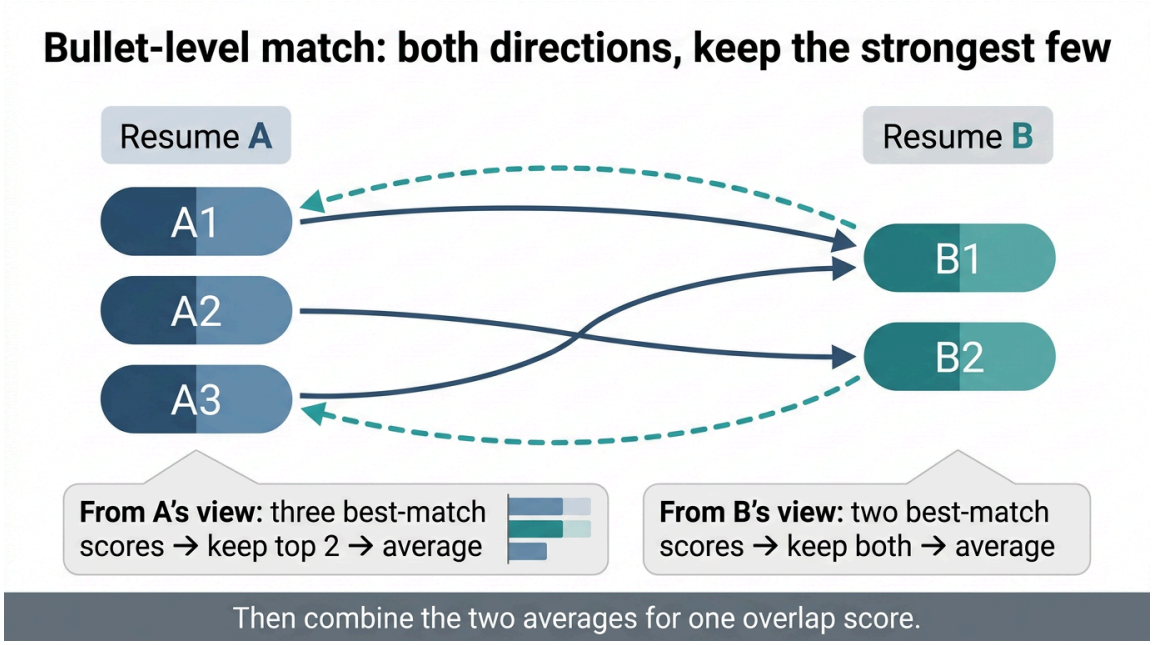
In [17]:
```
from sklearn.metrics.pairwise import cosine_similarity

# --- tune experience row weights (RecencyMonthsAgo → α) ---
WEIGHT_EXPERIENCE_RECENT = 2.00
WEIGHT_EXPERIENCE_MID = 1.00
WEIGHT_EXPERIENCE_OLD = 0.5
RECENCY_MONTHS_AGO_MAX_RECENT = 12
RECENCY_MONTHS_AGO_MAX_MID = 48


def recency_weight(
    months_ago: float,
    w_recent: float,
    w_mid: float,
    w_old: float,
    max_recent: float,
    max_mid: float,
) -> float:
    if max_recent >= max_mid:
        raise ValueError(
            f"RECENCY_MONTHS_AGO_MAX_RECENT ({max_recent}) must be < "
            f"RECENCY_MONTHS_AGO_MAX_MID ({max_mid})"
        )
    if pd.isna(months_ago) or float(months_ago) <= max_recent:
        return w_recent
    if float(months_ago) <= max_mid:
        return w_mid
    return w_old


if not OUT_EMB_PATH.is_file():
    raise FileNotFoundError(f"Embedded corpus not found: {OUT_EMB_PATH.resolve()}")
if not META_PATH.is_file():
    raise FileNotFoundError(f"Resume metadata not found: {META_PATH.resolve()}")

emb_df2 = pd.read_csv(OUT_EMB_PATH, dtype={"ResumeID": int, "ItemID": int})
exp_mask = emb_df2["Section"].astype(str).str.strip() == "Experience"
exp_df = emb_df2.loc[exp_mask & emb_df2["embedding"].astype(str).str.len().gt(0)].c

all_resume_ids = sorted(emb_df2["ResumeID"].unique())
exp_counts = exp_df.groupby("ResumeID").size()
missing_exp = [rid for rid in all_resume_ids if rid not in exp_counts.index or exp_
if missing_exp:
    raise ValueError(f"No non-empty Experience embedding for ResumeID(s): {missing_
```

```
if exp_df["RecencyMonthsAgo"].isna().mean() > 0.5:
    print("Warning: over half of Experience rows have NaN RecencyMonthsAgo; those r

centroids: dict[int, np.ndarray] = {}
for rid in all_resume_ids:
    rows = exp_df.loc[exp_df["ResumeID"] == rid]
    w_list = []
    v_list = []
    for _, row in rows.iterrows():
        alpha = recency_weight(
            row["RecencyMonthsAgo"],
            WEIGHT_EXPERIENCE_RECENT,
            WEIGHT_EXPERIENCE_MID,
            WEIGHT_EXPERIENCE_OLD,
            RECENCY_MONTHS_AGO_MAX_RECENT,
            RECENCY_MONTHS_AGO_MAX_MID,
        )
        v = np.array(json.loads(row["embedding"]), dtype=np.float32)
        w_list.append(alpha)
        v_list.append(v * alpha)
    w_sum = float(np.sum(w_list))
    centroids[rid] = np.sum(np.stack(v_list, axis=0), axis=0) / w_sum

if TARGET_RESUME_ID not in centroids:
    raise ValueError(f"TARGET_RESUME_ID={TARGET_RESUME_ID} has no experience centro

ids_c = sorted(centroids.keys())
Xc = np.stack([centroids[rid] for rid in ids_c], axis=0)
target_c = centroids[TARGET_RESUME_ID].reshape(1, -1)
cent_sims = cosine_similarity(target_c, Xc)[0]

centroid_rank_df = (
    pd.DataFrame({"ResumeID": ids_c, "centroid_cosine_similarity": cent_sims.astype
    .sort_values("centroid_cosine_similarity", ascending=False)
    .reset_index(drop=True)
)
meta_c = pd.read_csv(META_PATH)
centroid_rank_df = centroid_rank_df.merge(meta_c, on="ResumeID", how="left")

ti = ids_c.index(TARGET_RESUME_ID)
self_c = float(cent_sims[ti])
assert abs(self_c - 1.0) < 1e-5, f"expected target centroid self-similarity ~1, got
assert cent_sims.min() >= -1.0001 and cent_sims.max() <= 1.0001

centroid_rank_df
```

Out[17]:

	ResumeID	centroid_cosine_similarity	primary_domain	role_family	seniority_
0	1	1.000000	data_analytics	healthcare_analytics	
1	4	0.679951	data_analytics	healthcare_reporting	
2	5	0.587586	data_analytics	product_analytics	
3	2	0.587265	data_analytics	bi_reporting	
4	9	0.546896	data_science	forecasting	
5	14	0.542166	finance	fpna	s
6	6	0.515930	data_science	ml_engineering	
7	7	0.511272	data_science	research_ds	s
8	3	0.508768	data_analytics	operations_analytics	s
9	10	0.495333	data_analytics	public_sector	
10	8	0.490805	data_science	nlp_search	
11	19	0.462811	marketing	performance_marketing	
12	13	0.457718	hr	total_rewards	
13	11	0.440845	hr	talent_acquisition	
14	20	0.385974	marketing	brand_content	
15	15	0.370307	finance	corporate_accounting	
16	16	0.347784	finance	internal_audit	
17	12	0.320755	hr	employee_relations	s
18	17	0.292066	engineering	structural	
19	18	0.283828	engineering	manufacturing_quality	s

Step 6: Bullet-level alignment (concrete overlap)



BulletMatchSim compares `TARGET_RESUME_ID` to each other resume using **Experience** embeddings only. For a pair **(T, R)** with **m** and **n** bullets, build **M** = `cosine_similarity(X_T, X_R)` with shape **(m, n)**. **T→R:** per target row, take the **max** over **R**; keep the **top k** of those **m** values (mean). **R→T:** per **R** column, **max** over **T**; mean of top **k** of **n** values. **BulletMatchSim = (T→R + R→T) / 2**.

Configurable k: `BULLET_TOPK_MIN`, `BULLET_TOPK_FRACTION_OF_MIN`, `BULLET_TOPK_MAX` at the top of the next cell; `k = min(max(MIN, floor(FRACTION * min(m,n))), MAX)`, then `k_eff = min(k, m)` or `min(k, n)` per direction. Invalid `MIN` / `MAX` raises `ValueError`.

Without one-to-one matching, **one generic bullet** on one side can be the best match for **many** lines on the other; treat this as **component 3** of a future composite. Optional later: top bullet pairs for audit.

Checks: target **vs self** score **≈ 1**; scores in **[-1, 1]**.

Step 3: Bullet-Level Alignment

Example resumes

Resume 1 (target)

- **A1** = `[1, 0, 0]`
- **A2** = `[0, 1, 0]`
- **A3** = `[0, 0, 1]`

Resume 2

- **B1** = `[1, 0, 0]`
- **B2** = `[0, 1, 1]`

---

1. Compare every bullet to every bullet

This creates a **3 × 2 similarity matrix**:

	B1	B2
A1	1.00	0.00
A2	0.00	0.71
A3	0.00	0.71

Interpretation: each number shows how close two bullets are in meaning.

---

2. Pick the best match for each bullet

Resume 1 -> Resume 2

- A1 best match = **1.00**
- A2 best match = **0.71**
- A3 best match = **0.71**

Result: **[1.00, 0.71, 0.71]**

Resume 2 -> Resume 1

- B1 best match = **1.00**
- B2 best match = **0.71**

Result: **[1.00, 0.71]**

---

3. Calculate k

We choose **k** based on the smaller resume:

$$k = \min(\max(2, \lfloor 0.4 \times \min(m, n) \rfloor), 5)$$

Here:

- **m = 3**
- **n = 2**
- **min(3,2) = 2**
- **0.4 × 2 = 0.8**
- **floor(0.8) = 0**
- **max(2,0) = 2**
- **min(2,5) = 2**

k = 2

---

4. Keep the top 2 matches in each direction

Resume 1 -> Resume 2
Top 2 of `[1.00, 0.71, 0.71]` = **[1.00, 0.71]**

Average:

$$(1.00 + 0.71)/2 = 0.855$$

Resume 2 -> Resume 1
Top 2 of `[1.00, 0.71]` = **[1.00, 0.71]**

Average:

$$(1.00 + 0.71)/2 = 0.855$$

---

5. Final bullet alignment score

$$\text{BulletMatchSim} = (0.855 + 0.855)/2 = 0.855$$

Final result: 0.855

Explain K

What “configurable k” means here k is how many strongest per-bullet matches you keep when turning a full bullet–bullet matrix into one number per direction.

For each direction you first compute many scores, then you do not average all of them. You:

Take one score per bullet on that side (its best match on the other side). Sort those scores high → low. Keep only the top k (after capping—see below) and take their mean. So k answers: “How much of the best overlap should count, before longer resumes dilute the signal?”

The plan makes k “configurable” by driving it from three knobs at the top of the cell (defaults in the plan):

BULLET_TOPK_MIN (e.g. 2) — never use fewer than this in the formula’s inner max (unless the resume has fewer bullets than that—then you use what you have).
BULLET_TOPK_FRACTION_OF_MIN (e.g. 0.4) — scale k with min(m, n), where m = target

bullet count, n = other resume's bullet count (for that pair). BULLET_TOPK_MAX (e.g. 5) — cap k so it never gets huge on long resumes. Formula (as in the plan):

k = min(max(BULLET_TOPK_MIN, floor(FRACTION * min(m, n))), BULLET_TOPK_MAX)

Then when taking the mean of "top k" scores:

T → R: you have m scores (one per target bullet). Use k_eff = min(k, m) scores. R → T: you have n scores (one per R bullet). Use k_eff2 = min(k, n) scores. "Configurable" means: you change BULLET_TOPK_MIN, BULLET_TOPK_FRACTION_OF_MIN, or BULLET_TOPK_MAX to make k larger or smaller for the same (m, n)—tuning how much you emphasize peak overlap vs breadth.

Your example: A has 10 bullets, B has 5 Fix which resume is the target (call it T). The other is R.

Case 1 — Target is A (10 bullets), other is B (5) Then m = 10, n = 5, min(m, n) = 5.

With defaults MIN=2, FRACTION=0.4, MAX=5:

floor(0.4 * 5) = 2

max(2, 2) = 2

min(2, 5) = 2 So k = 2.

A → B: for each of 10 A-bulits, best cosine to some B-bullet → 10 numbers; keep the top 2 of those 10; mean of those 2.

B → A: for each of 5 B-bulits, best cosine to some A-bullet → 5 numbers; keep the top 2 of those 5; mean of those 2.

BulletMatchSim(A, B) = average of those two means.

Case 2 — Target is B (5 bullets), other is A (10) Still min(10, 5) = 5, so k = 2 again. The matrix is 5×10 instead of 10×5, but min(m,n) is the same, so k is the same. The two directions still use m and n correctly for how many "per-bullet best" scores exist on each side.

Intuition Smaller k (e.g. sticking near 2 for this pair): the score is driven mostly by the strongest alignments; weak bullets barely move it. Larger k (raise FRACTION or MAX): you average more of the per-bullet bests, so the score reflects more of the resume's bullets, not only the sharpest peaks. So for 10 vs 5, with default knobs, k = 2 is the planned behavior for both directions' top-k means before they are combined.

In [19]:
```
import math

from sklearn.metrics.pairwise import cosine_similarity

# --- top-k size for bidirectional bullet aggregation ---
```

```
BULLET_TOPK_MIN = 2
BULLET_TOPK_MAX = 5
BULLET_TOPK_FRACTION_OF_MIN = 0.4


def bullet_topk_k(m: int, n: int, min_k: int, max_k: int, fraction: float) -> int:
    if min_k < 1:
        raise ValueError("BULLET_TOPK_MIN must be >= 1")
    if max_k < min_k:
        raise ValueError("BULLET_TOPK_MAX must be >= BULLET_TOPK_MIN")
    mn = min(m, n)
    return int(min(max(min_k, math.floor(fraction * mn)), max_k))


def mean_of_top_k_largest(scores_1d: np.ndarray, k: int) -> float:
    if scores_1d.size == 0:
        raise ValueError("empty bullet score vector")
    k_eff = min(k, int(scores_1d.size))
    desc = np.sort(scores_1d.astype(np.float64))[::-1][:k_eff]
    return float(np.mean(desc))


if not OUT_EMB_PATH.is_file():
    raise FileNotFoundError(f"Embedded corpus not found: {OUT_EMB_PATH.resolve()}")
if not META_PATH.is_file():
    raise FileNotFoundError(f"Resume metadata not found: {META_PATH.resolve()}")

emb_df3 = pd.read_csv(OUT_EMB_PATH, dtype={"ResumeID": int, "ItemID": int})
exp_mask3 = emb_df3["Section"].astype(str).str.strip() == "Experience"
exp_b = emb_df3.loc[exp_mask3 & emb_df3["embedding"].astype(str).str.len().gt(0)].c
exp_b = exp_b.sort_values(["ResumeID", "ItemID"], kind="mergesort")

all_ids_b = sorted(emb_df3["ResumeID"].unique())
exp_counts_b = exp_b.groupby("ResumeID").size()
missing_b = [rid for rid in all_ids_b if rid not in exp_counts_b.index or exp_count
if missing_b:
    raise ValueError(f"No non-empty Experience embedding for ResumeID(s): {missing_

bullets_by_rid: dict[int, np.ndarray] = {}
for rid, grp in exp_b.groupby("ResumeID", sort=False):
    vecs = [np.array(json.loads(e), dtype=np.float32) for e in grp["embedding"]]
    bullets_by_rid[int(rid)] = np.stack(vecs, axis=0)

if TARGET_RESUME_ID not in bullets_by_rid:
    raise ValueError(f"TARGET_RESUME_ID={TARGET_RESUME_ID} has no experience bullet

ids_b = sorted(bullets_by_rid.keys())
X_T = bullets_by_rid[TARGET_RESUME_ID]

rows_bm = []
for rid in ids_b:
    X_R = bullets_by_rid[rid]
    m_tgt, n_other = X_T.shape[0], X_R.shape[0]
    k = bullet_topk_k(m_tgt, n_other, BULLET_TOPK_MIN, BULLET_TOPK_MAX, BULLET_TOPK
    M = cosine_similarity(X_T, X_R)
    t_to_r = mean_of_top_k_largest(np.max(M, axis=1), k)
```

```
    r_to_t = mean_of_top_k_largest(np.max(M, axis=0), k)
    bullet_sim = (t_to_r + r_to_t) / 2.0
    rows_bm.append({"ResumeID": rid, "bullet_match_similarity": bullet_sim})

self_bm = float(next(r["bullet_match_similarity"] for r in rows_bm if r["ResumeID"]

bullet_rank_df = (
    pd.DataFrame(rows_bm)
    .sort_values("bullet_match_similarity", ascending=False)
    .reset_index(drop=True)
)
meta_b = pd.read_csv(META_PATH)
bullet_rank_df = bullet_rank_df.merge(meta_b, on="ResumeID", how="left")

sims_bm = bullet_rank_df["bullet_match_similarity"].to_numpy(dtype=np.float64)
assert abs(self_bm - 1.0) < 1e-5, f"expected target bullet self-similarity ~1, got
assert sims_bm.min() >= -1.0001 and sims_bm.max() <= 1.0001

bullet_rank_df
```

Out[19]:

	ResumeID	bullet_match_similarity	primary_domain	role_family	seniority_ba
0	1	1.000000	data_analytics	healthcare_analytics	r
1	4	0.525314	data_analytics	healthcare_reporting	ea
2	7	0.514245	data_science	research_ds	ser
3	2	0.461316	data_analytics	bi_reporting	r
4	5	0.460672	data_analytics	product_analytics	r
5	3	0.454597	data_analytics	operations_analytics	ser
6	6	0.441418	data_science	ml_engineering	r
7	9	0.426676	data_science	forecasting	r
8	13	0.394523	hr	total_rewards	r
9	14	0.385076	finance	fpna	ser
10	19	0.383509	marketing	performance_marketing	r
11	10	0.376920	data_analytics	public_sector	ea
12	11	0.375893	hr	talent_acquisition	r
13	8	0.363487	data_science	nlp_search	r
14	12	0.351520	hr	employee_relations	ser
15	15	0.323798	finance	corporate_accounting	r
16	20	0.319476	marketing	brand_content	ea
17	16	0.285311	finance	internal_audit	ea
18	18	0.276351	engineering	manufacturing_quality	ser
19	17	0.259549	engineering	structural	r

Step 7: Composite score (vs `TARGET_RESUME_ID`)

Merge the three signals

FinalScore(T, R) = w_1·SummarySim + w_2·CentroidSim + w_3·BulletMatchSim with **T =** `TARGET_RESUME_ID`.

Tune `W_SUMMARY`, `W_CENTROID`, `W_BULLET` at the top of the next cell (defaults **0.20 / 0.30 / 0.50**); they must **sum to 1** (or the cell raises). Output `composite_rank_df`: `final_score` plus the three component columns and metadata.

Run Steps 1–3 first so `rank_df`, `centroid_rank_df`, and `bullet_rank_df` exist.

Step 7a: Top bullet pairs (evidence)

For the **top** `EVIDENCE_TOP_RESUMES` neighbors by `final_score` (excluding self), list the `EVIDENCE_BULLET_PAIRS` strongest **matrix cells (one target Experience row × one other Experience row)** by raw cosine similarity. So `pair_rank_in_resume_pair` 1…K means "K-th strongest *pair*," **not** "other resume's ItemID 1…K." The table includes **full bullet text for the matching (other) resume** plus the target line for context. Same caveat as Step 3: one generic bullet on either side can dominate multiple pairs. Output `bullet_evidence_df`.

In [20]:
```
# --- composite weights (must sum to 1) ---
W_SUMMARY = 0.2
W_CENTROID = 0.3
W_BULLET = 0.5

_wsum = W_SUMMARY + W_CENTROID + W_BULLET
if abs(_wsum - 1.0) > 1e-6:
    raise ValueError(f"W_SUMMARY + W_CENTROID + W_BULLET must equal 1; got {_wsum}"

for _df, _col, _step in (
    (rank_df, "summary_cosine_similarity", "Step 1"),
    (centroid_rank_df, "centroid_cosine_similarity", "Step 2"),
    (bullet_rank_df, "bullet_match_similarity", "Step 3"),
):
    if _col not in _df.columns:
        raise RuntimeError(f"Missing {_col}; run {_step} cell first.")

composite_df = (
    rank_df[["ResumeID", "summary_cosine_similarity"]]
    .merge(centroid_rank_df[["ResumeID", "centroid_cosine_similarity"]], on="Resume
    .merge(bullet_rank_df[["ResumeID", "bullet_match_similarity"]], on="ResumeID")
)
composite_df["final_score"] = (
    W_SUMMARY * composite_df["summary_cosine_similarity"]
    + W_CENTROID * composite_df["centroid_cosine_similarity"]
    + W_BULLET * composite_df["bullet_match_similarity"]
)

meta_f = pd.read_csv(META_PATH)
composite_rank_df = (
    composite_df.merge(meta_f, on="ResumeID", how="left")
    .sort_values("final_score", ascending=False)
    .reset_index(drop=True)
)

_self_final = float(
    composite_rank_df.loc[composite_rank_df["ResumeID"] == TARGET_RESUME_ID, "final
)
assert abs(_self_final - 1.0) < 1e-5, f"expected target final_score ~1, got {_self_
_fs = composite_rank_df["final_score"].to_numpy(dtype=np.float64)
assert _fs.min() >= -1.0001 and _fs.max() <= 1.0001

composite_rank_df
```

Out[20]:

	ResumeID	summary_cosine_similarity	centroid_cosine_similarity	bullet_match_similarity
0	1	1.000000	1.000000	1.000000
1	4	0.515133	0.679951	0.525314
2	7	0.499845	0.511272	0.514245
3	2	0.406697	0.587265	0.461316
4	5	0.257279	0.587586	0.460672
5	9	0.361967	0.546896	0.426676
6	3	0.330743	0.508768	0.454597
7	14	0.335894	0.542166	0.385076
8	6	0.233892	0.515930	0.441418
9	8	0.444134	0.490805	0.363487
10	13	0.329266	0.457718	0.394523
11	10	0.240335	0.495333	0.376920
12	19	0.239304	0.462811	0.383509
13	11	0.254773	0.440845	0.375893
14	20	0.258688	0.385974	0.319476
15	15	0.219156	0.370307	0.323798
16	12	0.150982	0.320755	0.351520
17	16	0.250858	0.347784	0.285311
18	17	0.265207	0.292066	0.259549
19	18	0.030978	0.283828	0.276351

In [21]:

```
from sklearn.metrics.pairwise import cosine_similarity

try:
    from IPython.display import display
except ImportError:
    display = print  # noqa: A001

# --- Step 4a: bullet text evidence for top neighbors by final_score ---
EVIDENCE_TOP_RESUMES = 5
EVIDENCE_BULLET_PAIRS = 5
# Short preview for the *target* bullet only; the matching resume uses full text in
TARGET_BULLET_SNIPPET_MAX_CHARS = 200

if "composite_rank_df" not in dir():
    raise RuntimeError("Run the Step 4 composite cell first (composite_rank_df).")
```

```python
if not OUT_EMB_PATH.is_file():
    raise FileNotFoundError(f"Embedded corpus not found: {OUT_EMB_PATH.resolve()}")

emb_ev = pd.read_csv(OUT_EMB_PATH, dtype={"ResumeID": int, "ItemID": int})
emb_ev["Title"] = emb_ev["Title"].fillna("").astype(str)
exp_mask_ev = emb_ev["Section"].astype(str).str.strip() == "Experience"
exp_ev = emb_ev.loc[exp_mask_ev & emb_ev["embedding"].astype(str).str.len().gt(0)].
exp_ev = exp_ev.sort_values(["ResumeID", "ItemID"], kind="mergesort")


def _full_bullet_label(title: str, text: str) -> str:
    title = (title or "").strip()
    text = (text or "").strip()
    return f"{title}: {text}" if title else text


def _maybe_shorten(label: str, max_chars: int) -> str:
    if len(label) <= max_chars:
        return label
    return label[: max_chars - 1] + "..."


records_ev: dict[int, list[dict]] = {}
for rid, grp in exp_ev.groupby("ResumeID", sort=False):
    recs = []
    for _, row in grp.iterrows():
        full_lbl = _full_bullet_label(row["Title"], row["Text"])
        recs.append(
            {
                "ItemID": int(row["ItemID"]),
                "bullet_text_full": full_lbl,
                "bullet_label_short": _maybe_shorten(full_lbl, TARGET_BULLET_SNIPPE
                "vec": np.array(json.loads(row["embedding"]), dtype=np.float32),
            }
        )
    records_ev[int(rid)] = recs

if TARGET_RESUME_ID not in records_ev:
    raise ValueError(f"TARGET_RESUME_ID={TARGET_RESUME_ID} has no experience rows f

neighbor_ids_ev = (
    composite_rank_df.loc[composite_rank_df["ResumeID"] != TARGET_RESUME_ID]
    .head(EVIDENCE_TOP_RESUMES)["ResumeID"]
    .astype(int)
    .tolist()
)

T_recs = records_ev[TARGET_RESUME_ID]
X_T_ev = np.stack([r["vec"] for r in T_recs], axis=0)

evidence_rows_ev: list[dict] = []
for other_rid in neighbor_ids_ev:
    R_recs = records_ev[other_rid]
    X_R_ev = np.stack([r["vec"] for r in R_recs], axis=0)
    M_ev = cosine_similarity(X_T_ev, X_R_ev)
    flat = M_ev.ravel()
```

```
    top_lin = np.argsort(flat, kind="mergesort")[::-1][:EVIDENCE_BULLET_PAIRS]
    ii, jj = np.unravel_index(top_lin, M_ev.shape)
    for rank, (lin, i, j) in enumerate(zip(top_lin, ii, jj), start=1):
        ti, rj = T_recs[int(i)], R_recs[int(j)]
        evidence_rows_ev.append(
            {
                "other_ResumeID": other_rid,
                "pair_rank_in_resume_pair": rank,
                "cosine_sim": float(flat[lin]),
                "other_ItemID": rj["ItemID"],
                "matching_resume_bullet_text": rj["bullet_text_full"],
                "target_ItemID": ti["ItemID"],
                "target_bullet_preview": ti["bullet_label_short"],
            }
        )

bullet_evidence_df = pd.DataFrame(evidence_rows_ev)
_prev_mcw = pd.get_option("display.max_colwidth")
pd.set_option("display.max_colwidth", None)
try:
    display(bullet_evidence_df)
finally:
    pd.set_option("display.max_colwidth", _prev_mcw)
```

	other_ResumeID	pair_rank_in_resume_pair	cosine_sim	other_ItemID	matching_resume_bu
0	4	1	0.534068	3	Analytics Intern: V readmission trends ir supported month committee
1	4	2	0.516560	2	Healthcare Data Pulled HEDIS-alignec in SQL; assisted clinica with numerator/den logic and au docum
2	4	3	0.498261	2	Healthcare Data Pulled HEDIS-alignec in SQL; assisted clinica with numerator/den logic and au docum
3	4	4	0.458536	2	Healthcare Data Pulled HEDIS-alignec in SQL; assisted clinica with numerator/den logic and au docum
4	4	5	0.440263	2	Healthcare Data Pulled HEDIS-alignec in SQL; assisted clinica with numerator/den logic and au docum
5	7	1	0.517807	3	Data Scientist: Bui journey models fro and EHR extracts; doc

	other_ResumeID	pair_rank_in_resume_pair	cosine_sim	other_ItemID	matching_resume_bu
					assumptions for r dis
6	7	2	0.510683	2	Senior Data Supported ada readouts; impl reproducible R work interim analyses und
7	7	3	0.446694	3	Data Scientist: Bui journey models fro and EHR extracts; doc assumptions for r dis
8	7	4	0.414888	3	Data Scientist: Bui journey models fro and EHR extracts; doc assumptions for r dis
9	7	5	0.393177	2	Senior Data Supported ada readouts; impl reproducible R work interim analyses und
10	2	1	0.469495	3	Reporting Analyst: Au weekly inventory e reports in SQL; partn supply chain t stockouts on t
11	2	2	0.453137	2	Senior BI Ana executive sales dash Power BI; standar definitions across reg

	other_ResumeID	pair_rank_in_resume_pair	cosine_sim	other_ItemID	matching_resume_bu
					cut ad-hoc repor finance
12	2	3	0.451530	2	Senior BI Ana executive sales dash Power BI; standar definitions across reg cut ad-hoc repor finance
13	2	4	0.378434	3	Reporting Analyst: Au weekly inventory e reports in SQL; partn supply chain t stockouts on t
14	2	5	0.367309	3	Reporting Analyst: Au weekly inventory e reports in SQL; partn supply chain t stockouts on t
15	5	1	0.530741	2	Product Analyst: Ow serve activatic defined events, bu dashboards, and sur experiment rea le
16	5	2	0.442397	2	Product Analyst: Ow serve activatic defined events, bu

	other_ResumeID	pair_rank_in_resume_pair	cosine_sim	other_ItemID	matching_resume_bu
					dashboards, and sur experiment rea le
17	5	3	0.368854	2	Product Analyst: Ow serve activatic defined events, bu dashboards, and sur experiment rea le
18	5	4	0.357187	2	Product Analyst: Ow serve activatic defined events, bu dashboards, and sur experiment rea le
19	5	5	0.355671	2	Product Analyst: Ow serve activatic defined events, bu dashboards, and sur experiment rea le
20	9	1	0.462751	3	Quantitative Ana Monte Carlo scer staffing under uncertainty; presente to operations le
21	9	2	0.401503	3	Quantitative Ana Monte Carlo scer staffing under uncertainty; presente to operations le

	other_ResumeID	pair_rank_in_resume_pair	cosine_sim	other_ItemID	matching_resume_bu
22	9	3	0.400263	3	Quantitative Ana Monte Carlo scer staffing under uncertainty; presente to operations le
23	9	4	0.379698	2	Data Scientist: spreadsheet forec hierarchical tir models; cut WAPE horizon SKU
24	9	5	0.378563	2	Data Scientist: spreadsheet forec hierarchical tir models; cut WAPE horizon SKU